Lab 6: Set Up Nexus Repository and Configure User Access

Student (GitHub username): amnajaved-dev1
Host: amnajaved-dev1@ubuntu
Nexus server: http://192.168.154.128:8006
Repository: DevOps **Directory:** Labs/Lab06

No Nexus password, admin.password value, gradle.properties or settings.xml appears in this document.

Task 1: Run Nexus Repository and Complete the Initial Setup

Nexus was started in Docker with the persistent nexus-data volume and ports 8006 (web) and 8007 (reserved for the Docker repository) published. The administrator password was replaced and anonymous access disabled during the setup wizard.

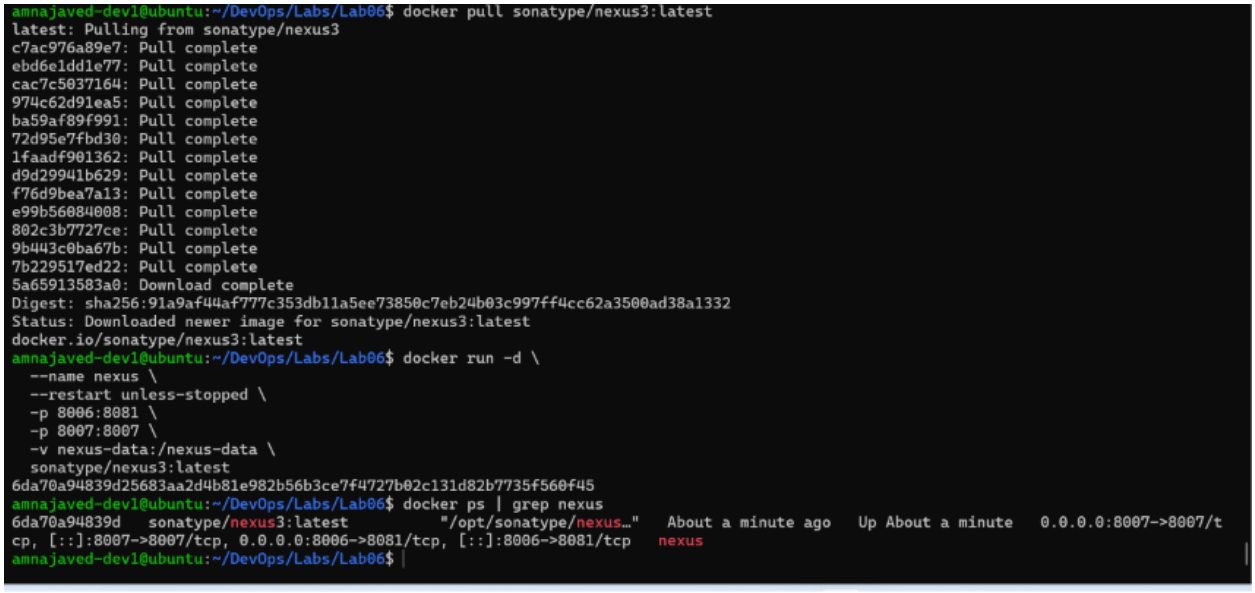


Figure 1. *nexus_container_running.png* — docker run creates the nexus container; docker ps | grep nexus shows it Up with ports 8006→8081 and 8007→8007 published.

Task 2: Create a Role and User and Verify Access

The nx-java role (privilege nx-repository-view-maven2-maven-snapshots-*) and the active local user nexus were created. The user was verified against the Nexus Status API.

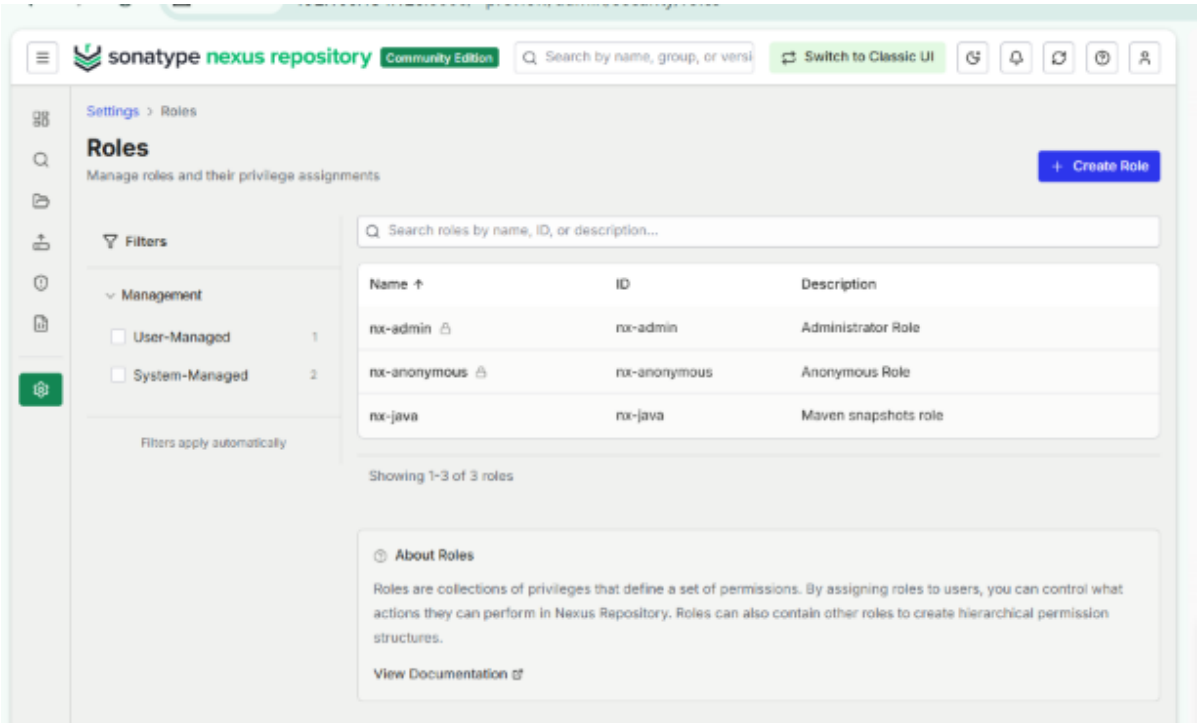


Figure 2. *nx_java_role.png* — Security → Roles: the saved nx-java role ("Maven snapshots role") alongside the system roles.

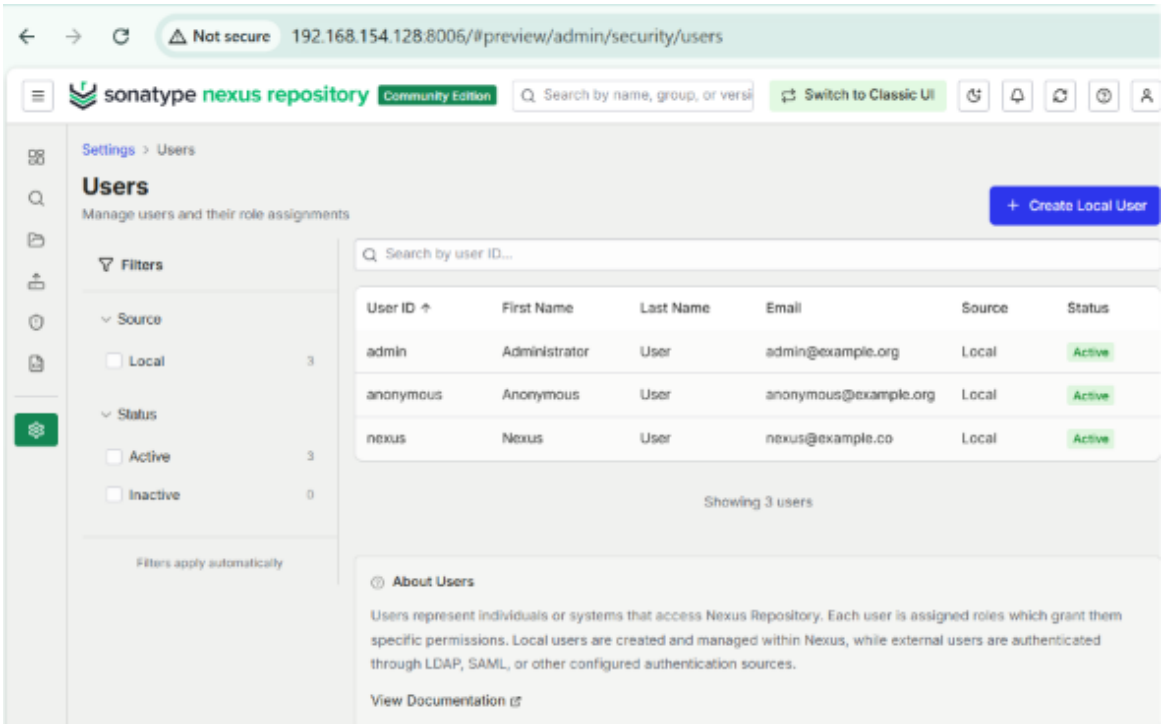
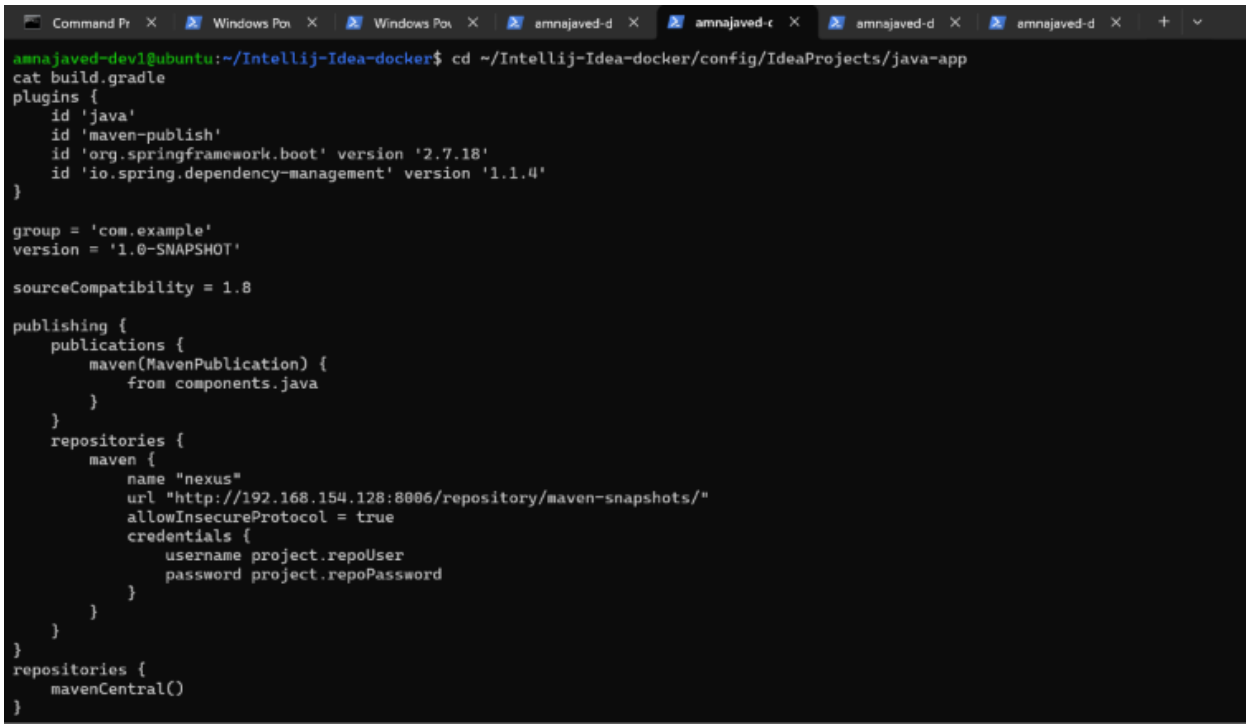


Figure 3. *nexus_user_access.png* — Security → Users: the local nexus user is Active and can authenticate to Nexus.

Task 3: Build and Publish the Gradle Application to Nexus

The java-app Gradle project was configured with the maven-publish plugin and the Nexus maven-snapshots repository. Credentials are read from the uncommitted gradle.properties file.



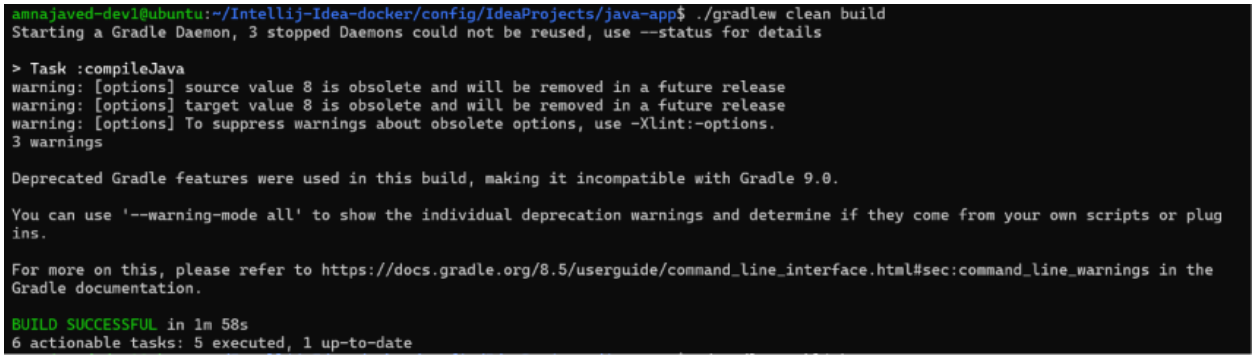
```
amnajaved-dev1@ubuntu:~/Intellij-Idea-docker$ cd ~/Intellij-Idea-docker/config/IdeaProjects/java-app
cat build.gradle
plugins {
    id 'java'
    id 'maven-publish'
    id 'org.springframework.boot' version '2.7.18'
    id 'io.spring.dependency-management' version '1.1.4'
}

group = 'com.example'
version = '1.0-SNAPSHOT'

sourceCompatibility = 1.8

publishing {
    publications {
        maven(MavenPublication) {
            from components.java
        }
    }
    repositories {
        maven {
            name "nexus"
            url "http://192.168.154.128:8806/repository/maven-snapshots/"
            allowInsecureProtocol = true
            credentials {
                username project.repoUser
                password project.repoPassword
            }
        }
    }
}
repositories {
    mavenCentral()
}
```

Figure 4. *gradle_nexus_configuration.png* — build.gradle with the maven-publish plugin, group/version and the publishing block pointing to Nexus maven-snapshots (credentials via project properties).



```
amnajaved-dev1@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-app$ ./gradlew clean build
Starting a Gradle Daemon, 3 stopped Daemons could not be reused, use --status for details

> Task :compileJava
warning: [options] source value 8 is obsolete and will be removed in a future release
warning: [options] target value 8 is obsolete and will be removed in a future release
warning: [options] To suppress warnings about obsolete options, use -Xlint:-options.
3 warnings

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plug
ins.

For more on this, please refer to https://docs.gradle.org/8.5/userguide/command_line_interface.html#sec:command_line_warnings in the
Gradle documentation.

BUILD SUCCESSFUL in 1m 58s
6 actionable tasks: 5 executed, 1 up-to-date
```

Figure 5. *gradle_nexus_build.png* — ./gradlew clean build finishes with BUILD SUCCESSFUL.



```
amnajaved-dev1@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-app$ ./gradlew publish

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plug
ins.

For more on this, please refer to https://docs.gradle.org/8.5/userguide/command_line_interface.html#sec:command_line_warnings in the
Gradle documentation.

BUILD SUCCESSFUL in 12s
6 actionable tasks: 3 executed, 3 up-to-date
amnajaved-dev1@ubuntu:~/Intellij-Idea-docker/config/IdeaProjects/java-app$
```

Figure 6. *gradle_nexus_publish.png* — Gradle publish to the Nexus repository finishes with BUILD SUCCESSFUL.

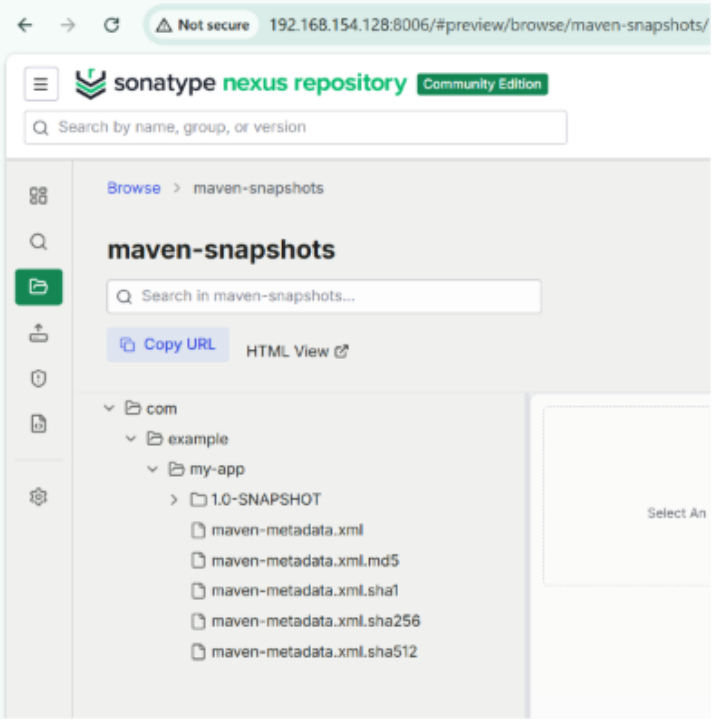


Figure 7. *nexus_gradle_artifact.png* — Nexus Browse → maven-snapshots → com/example/my-app showing the 1.0-SNAPSHOT folder and Maven metadata files.

Task 4: Publish the Maven Application to Nexus

The java-maven-app project was configured with distributionManagement and matching credentials in ~/.m2/settings.xml (not shown), packaged and deployed to Nexus.

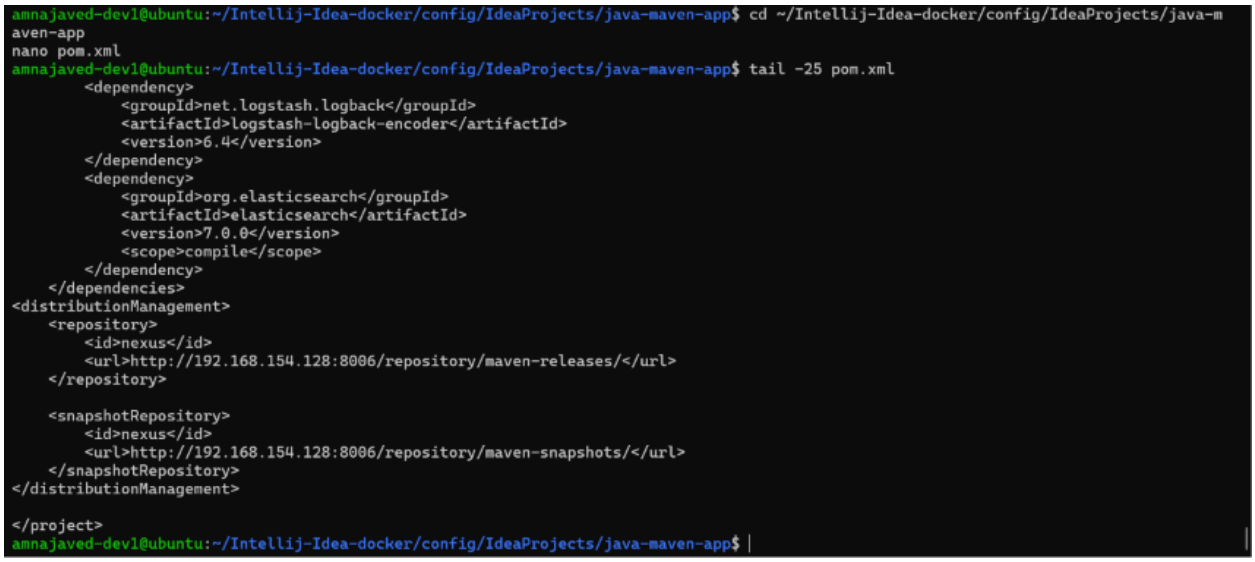


Figure 8. *maven_distribution_management.png* — pom.xml with the distributionManagement block, including the nexus-snapshots snapshotRepository.

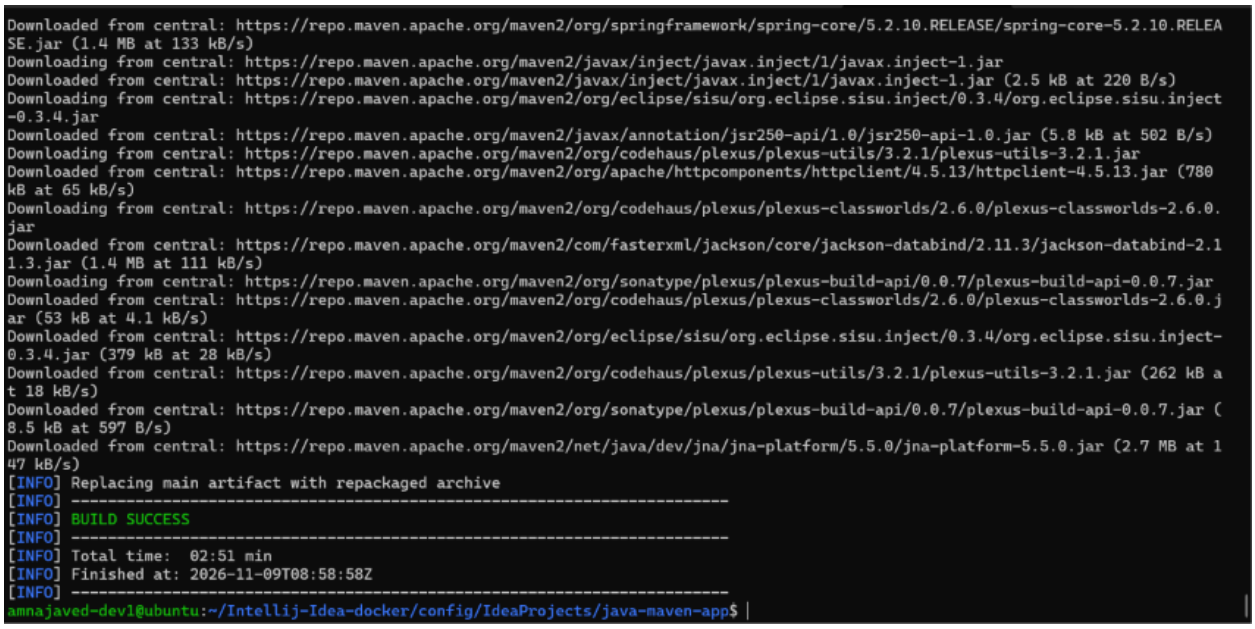


Figure 9. *maven_package_nexus.png* — mvn package finishes with BUILD SUCCESS.

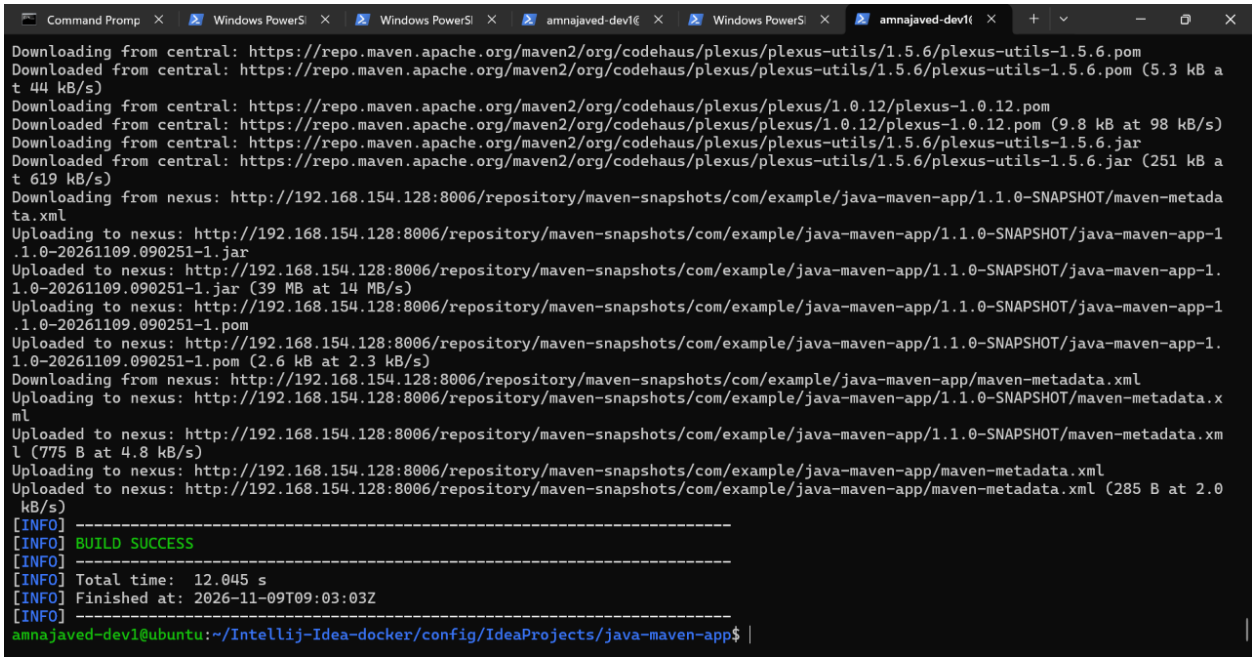


Figure 10. *maven_deploy_nexus.png* — mvn deploy uploads the JAR, POM and metadata to Nexus and finishes with BUILD SUCCESS.

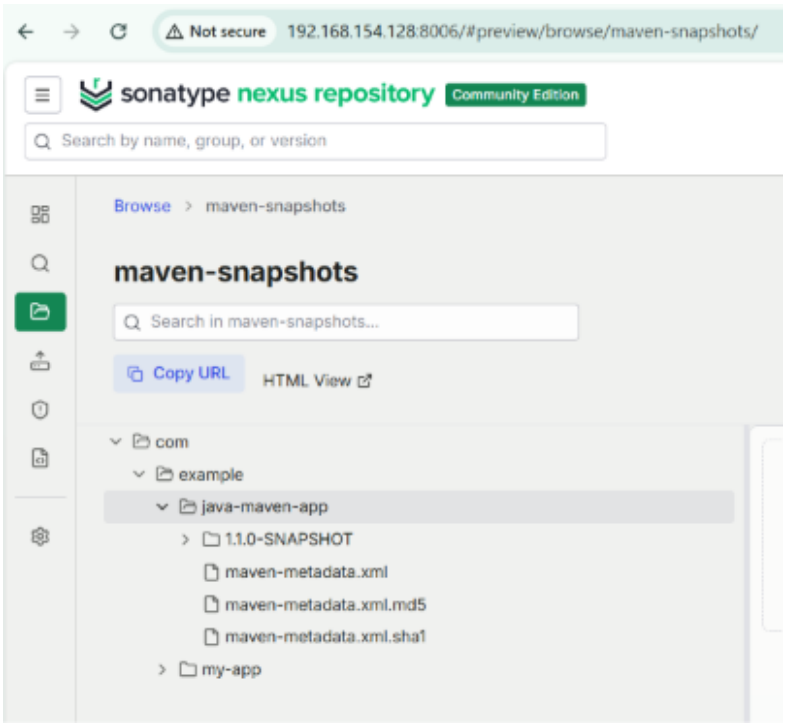


Figure 11. *nexus_maven_artifact.png* — Nexus Browse → maven-snapshots → com/example/java-maven-app showing 1.1.0-SNAPSHOT and Maven metadata.

Task 5: Inspect Nexus Repositories and Components Using the REST API

Repositories and components were queried through the Nexus REST API; curl prompted for passwords so none appear in commands.

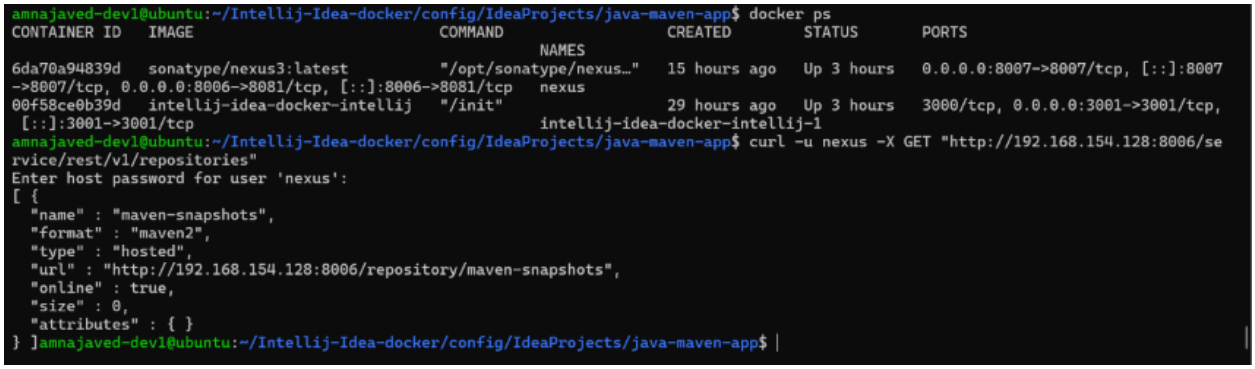


Figure 12. *nexus_repositories_user.png* — GET /service/rest/v1/repositories as the restricted nexus user.

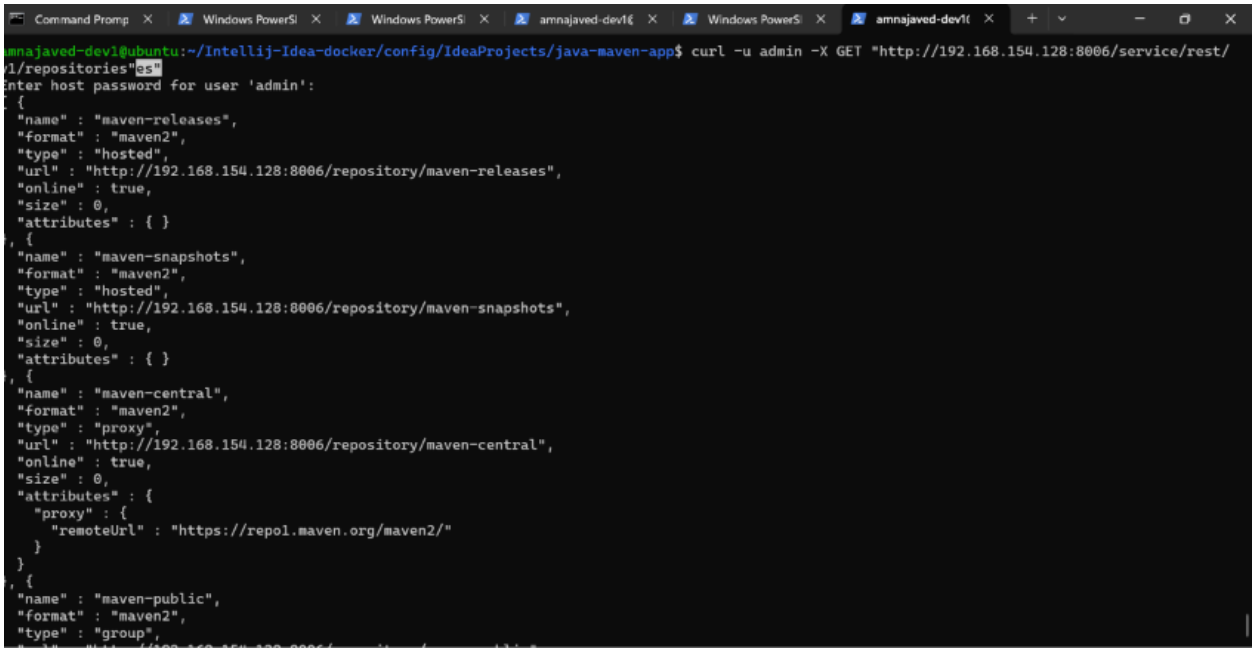


Figure 13. *nexus_repositories_admin.png* — GET /service/rest/v1/repositories as admin (compare with the restricted user's list).

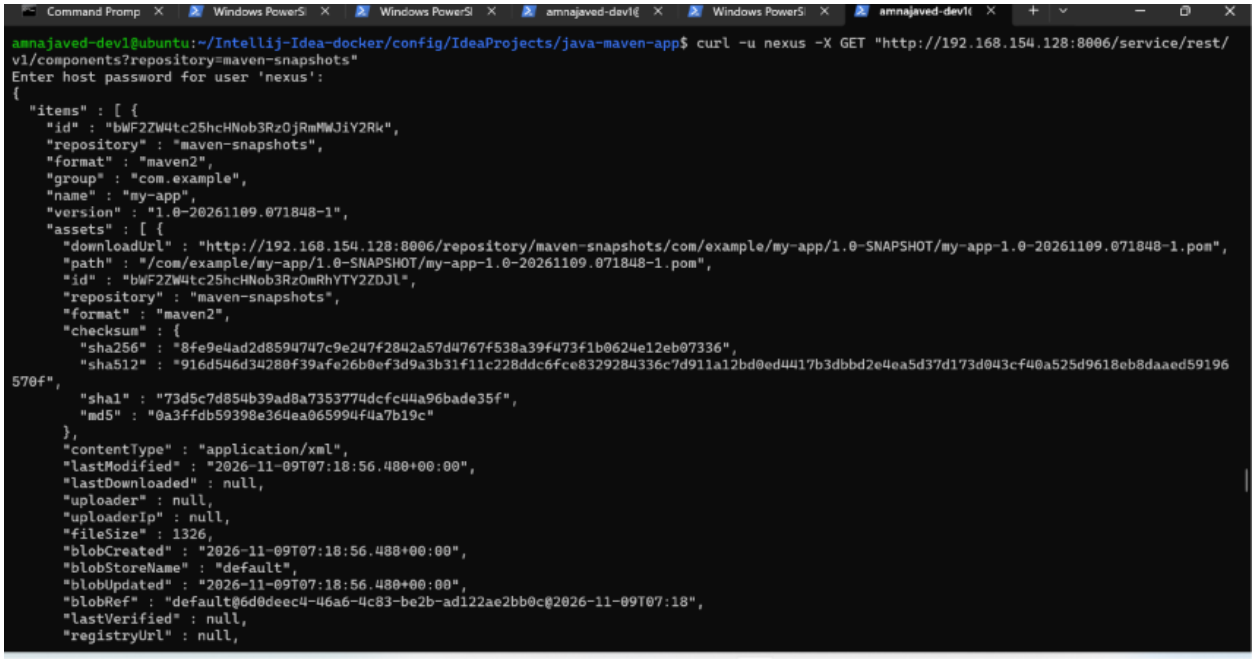


Figure 14. *nexus_components_list.png* — GET /service/rest/v1/components?repository=maven-snapshots showing a component's id, repository, name and version.

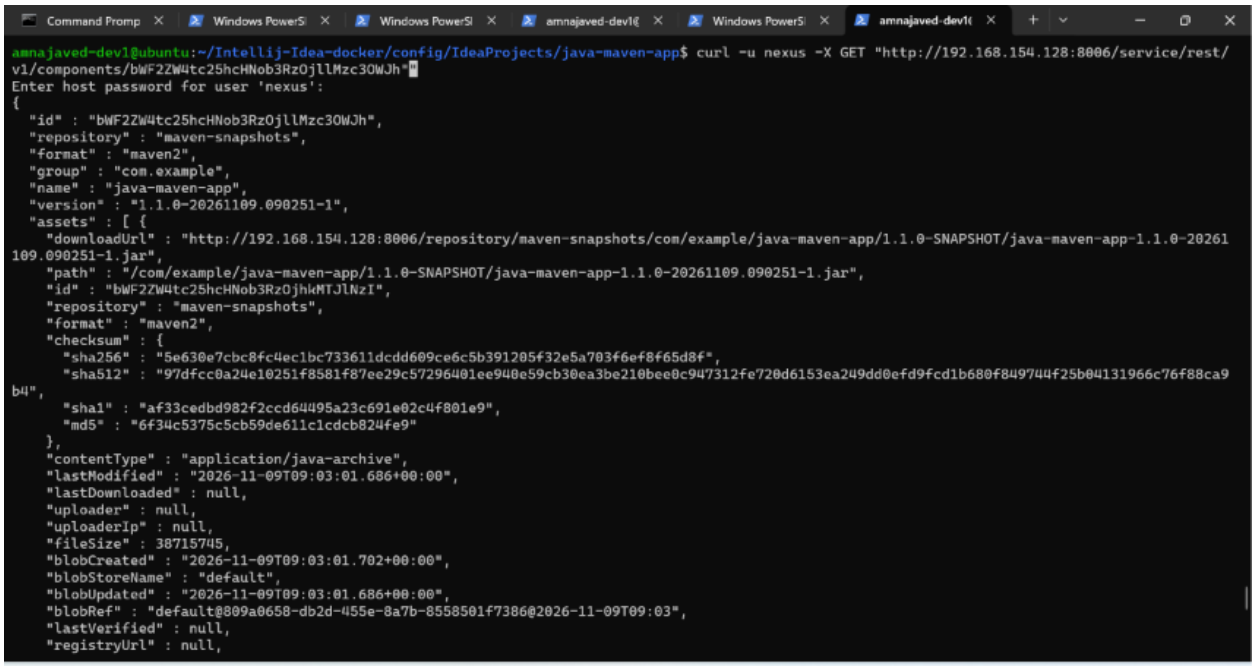


Figure 15. *nexus_component_details.png* — GET /service/rest/v1/components/ returning the selected component's details and assets.

Task 6: Create a Nexus Blob Store and Docker Hosted Repository

The default blob-store directory was inspected read-only, the mystore file blob store was created, and a docker-hosted repository on connector port 8007 was created using mystore.

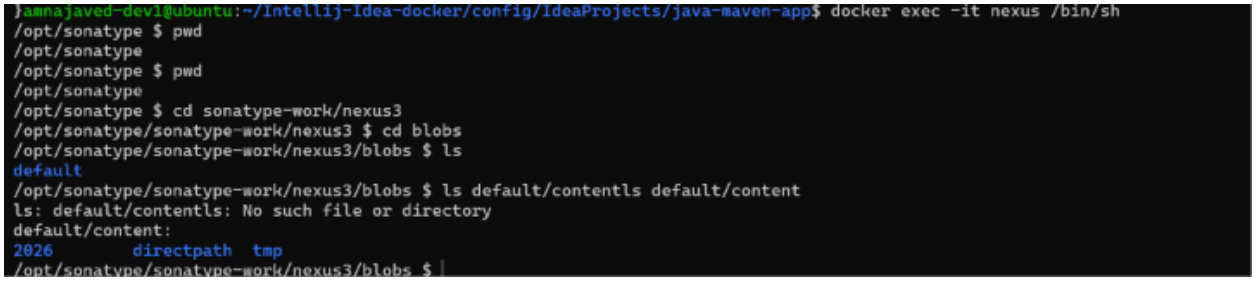


Figure 16. *nexus_default_blob_content.png* — docker exec into the nexus container; blobs directory, default blob store and its content listing.

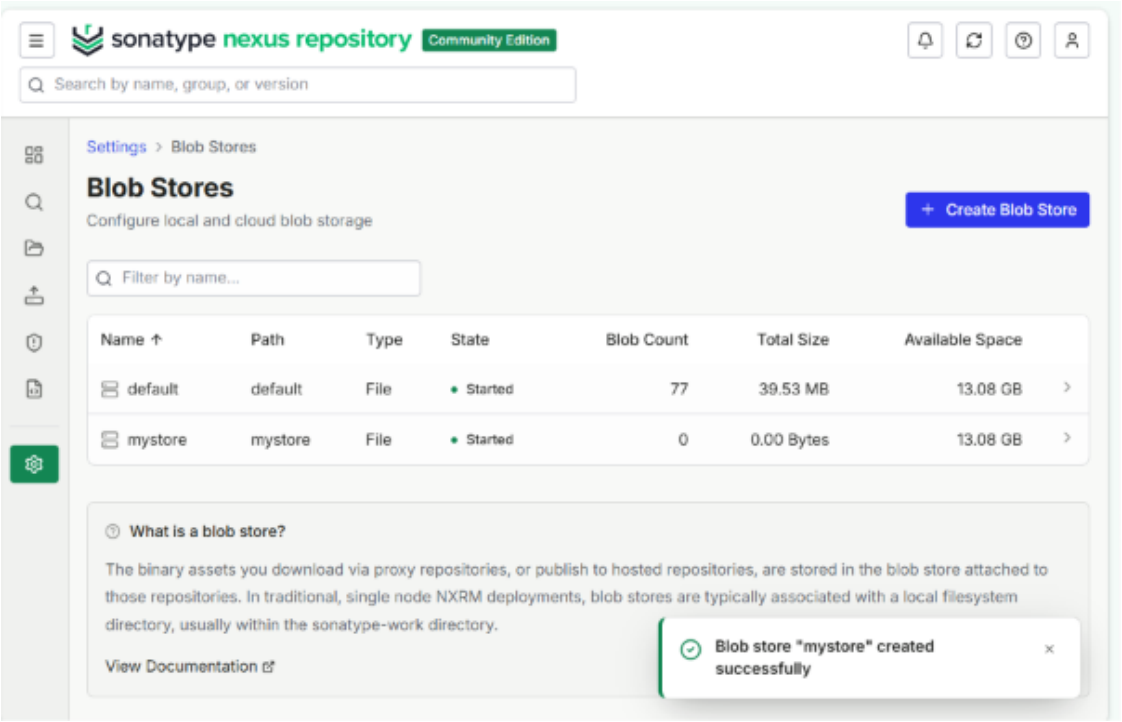


Figure 17. *nexus_mystore_created.png* — Blob Stores page showing default and mystore, both File type and Started.

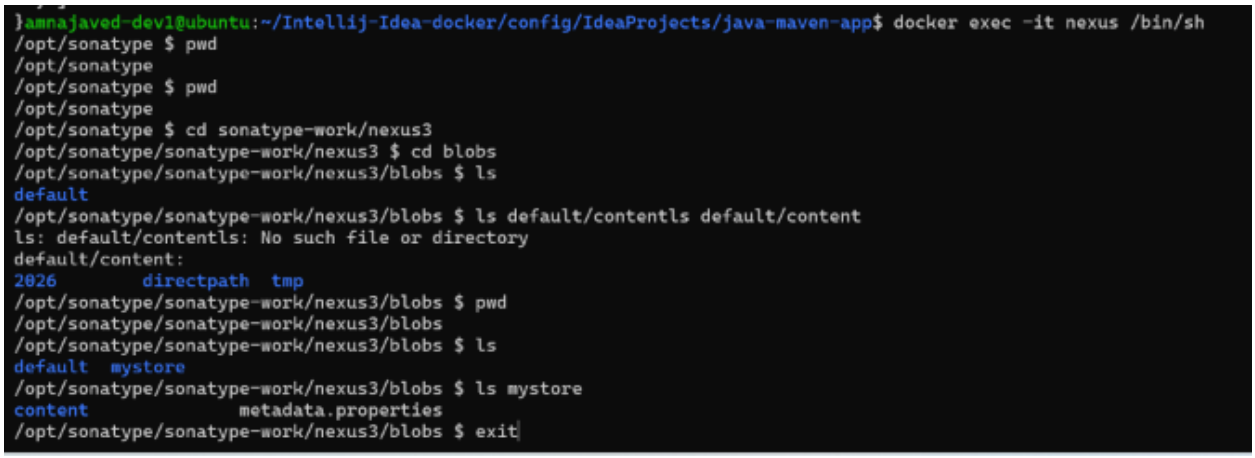


Figure 18. *nexus_blob_directories.png* — Container shell showing the blobs path with default and mystore directories, then exit back to the Ubuntu prompt.

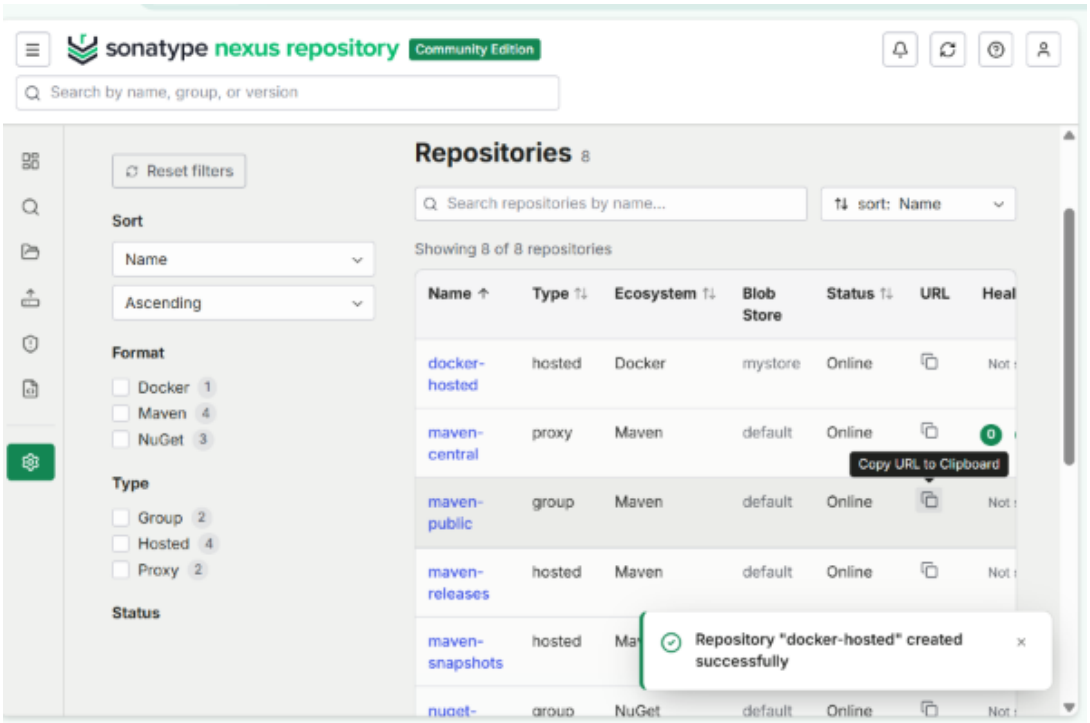


Figure 19. *nexus_docker_hosted_repository.png* — docker-hosted repository created: Online, docker/hosted, blob store mystore.

Task 7: Create and Run a Maven Snapshot Cleanup Policy

A cleanup policy matching all snapshot assets was created, attached to maven-snapshots and run; then the default blob store was compacted to reclaim disk space.

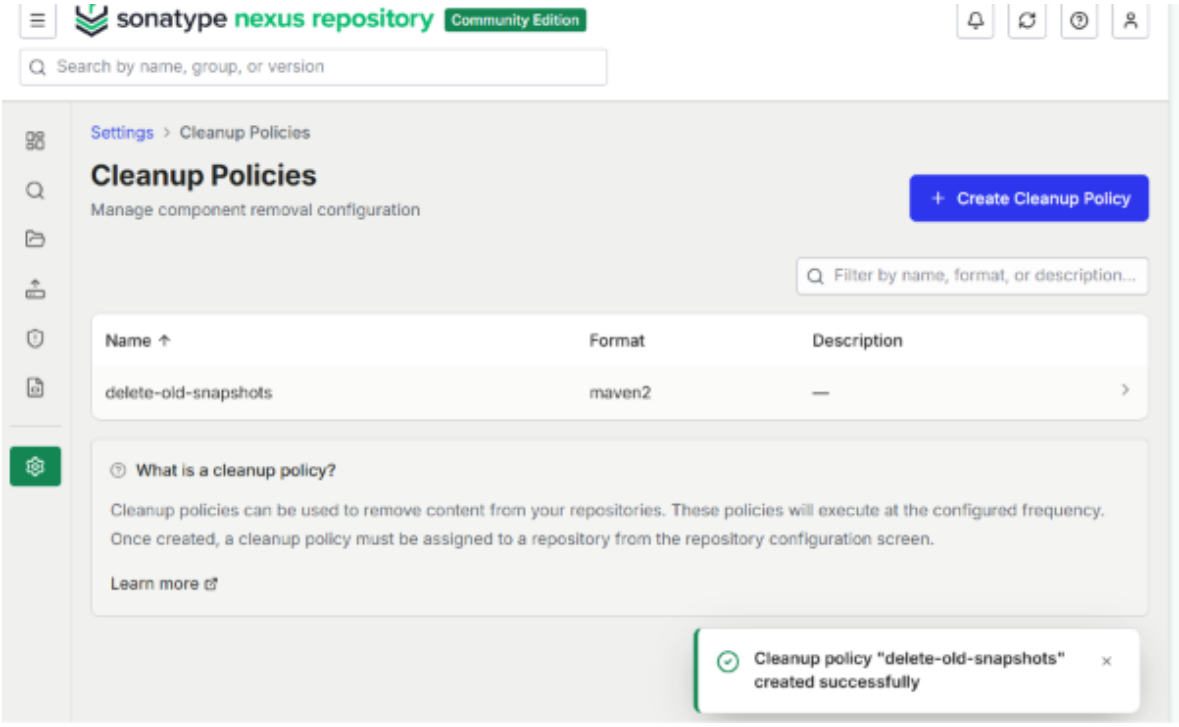


Figure 20. *nexus_cleanup_policy_created.png* — Cleanup policy "maven" (maven2, snapshots, .* matcher) with the maven-snapshots preview.

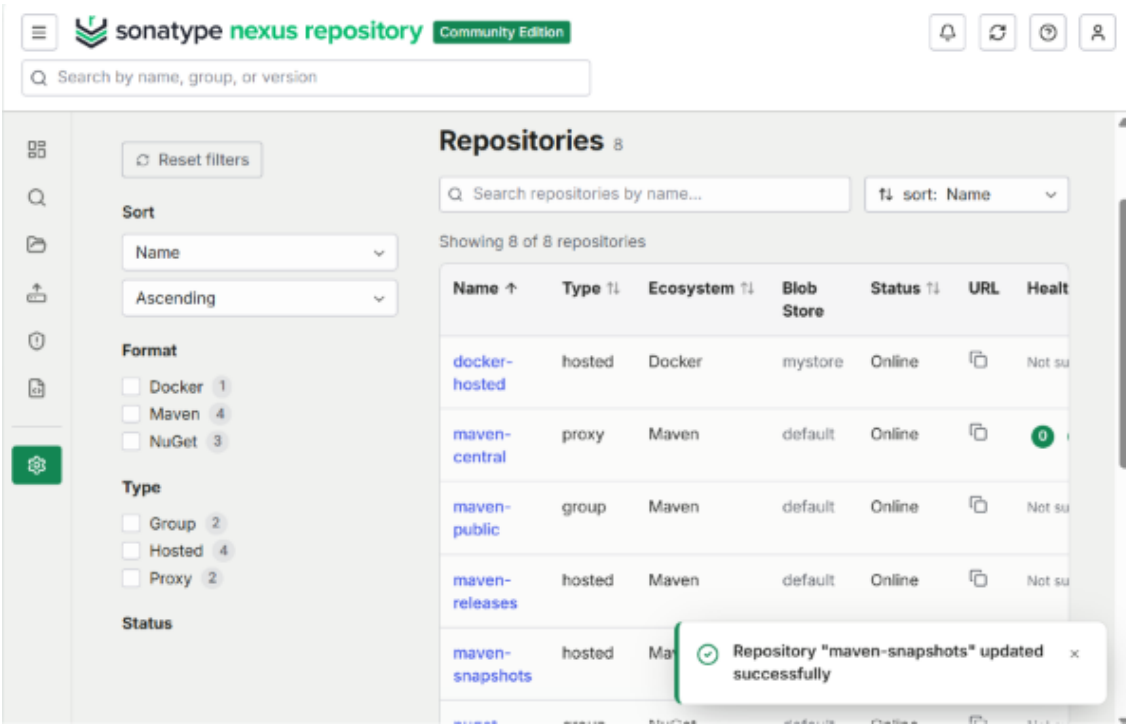


Figure 21. *nexus_cleanup_policy_attached.png* — maven-snapshots with the maven cleanup policy in the Applied list.

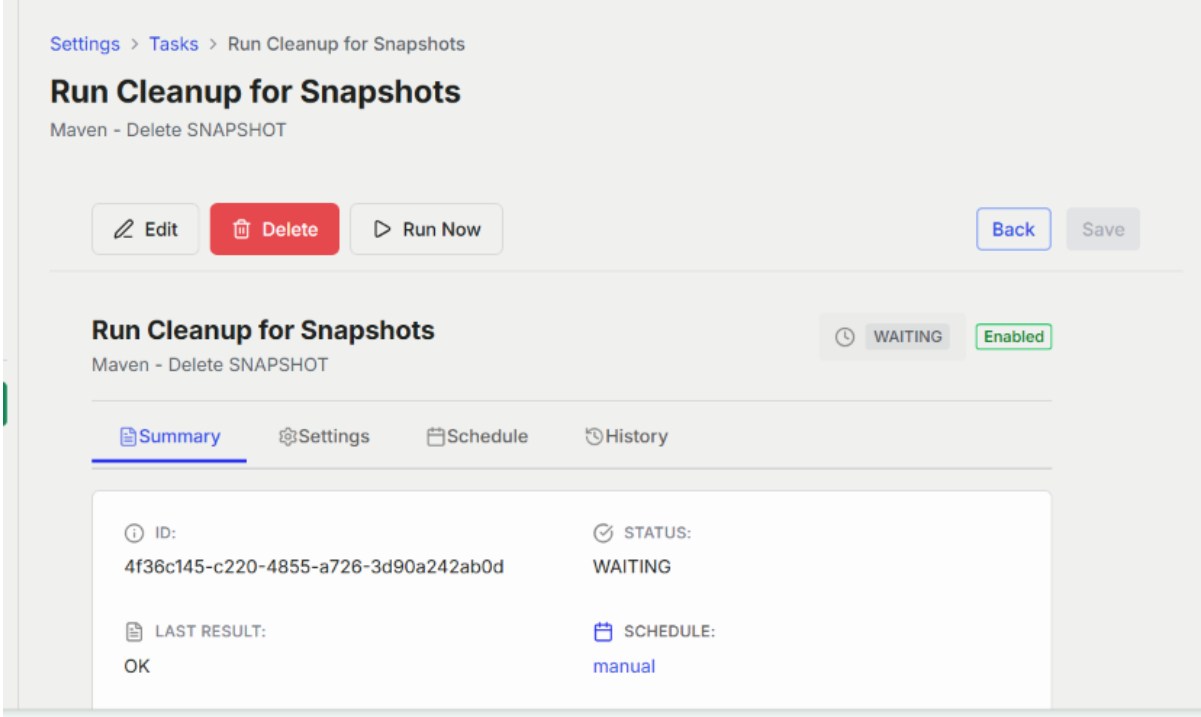


Figure 22. *nexus_cleanup_task_waiting.png* — Cleanup service task in Waiting state with the next run at 06:00.

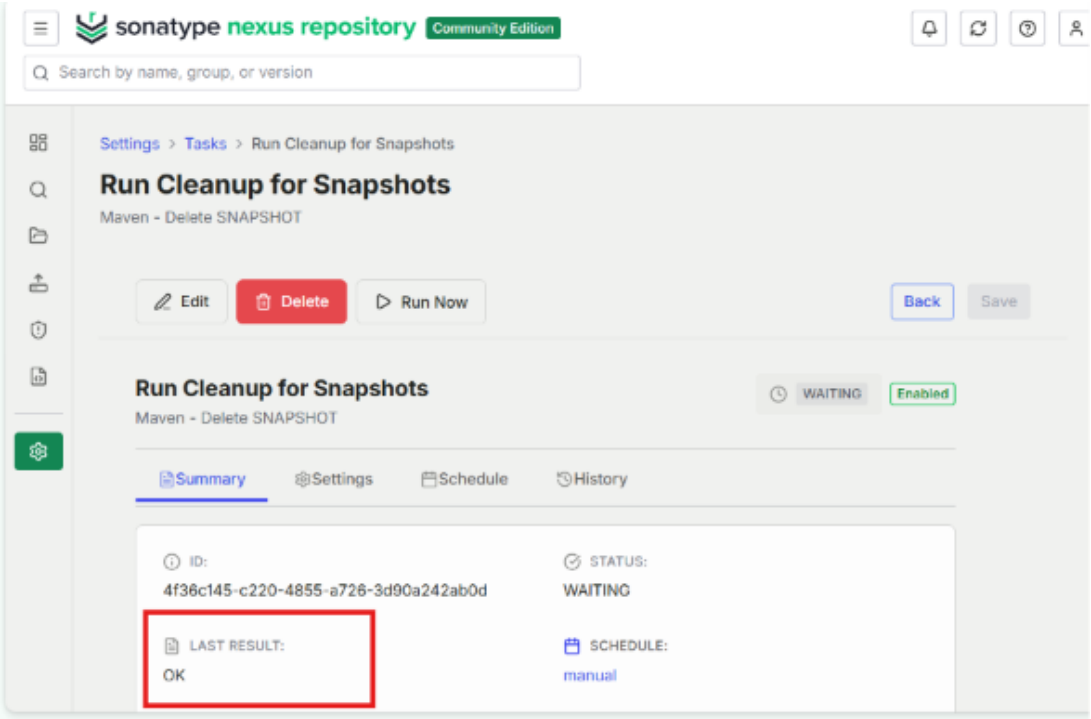


Figure 23. *nexus_cleanup_task_completed.png* — Cleanup service after the manual run: updated Last run and Last result: Ok.

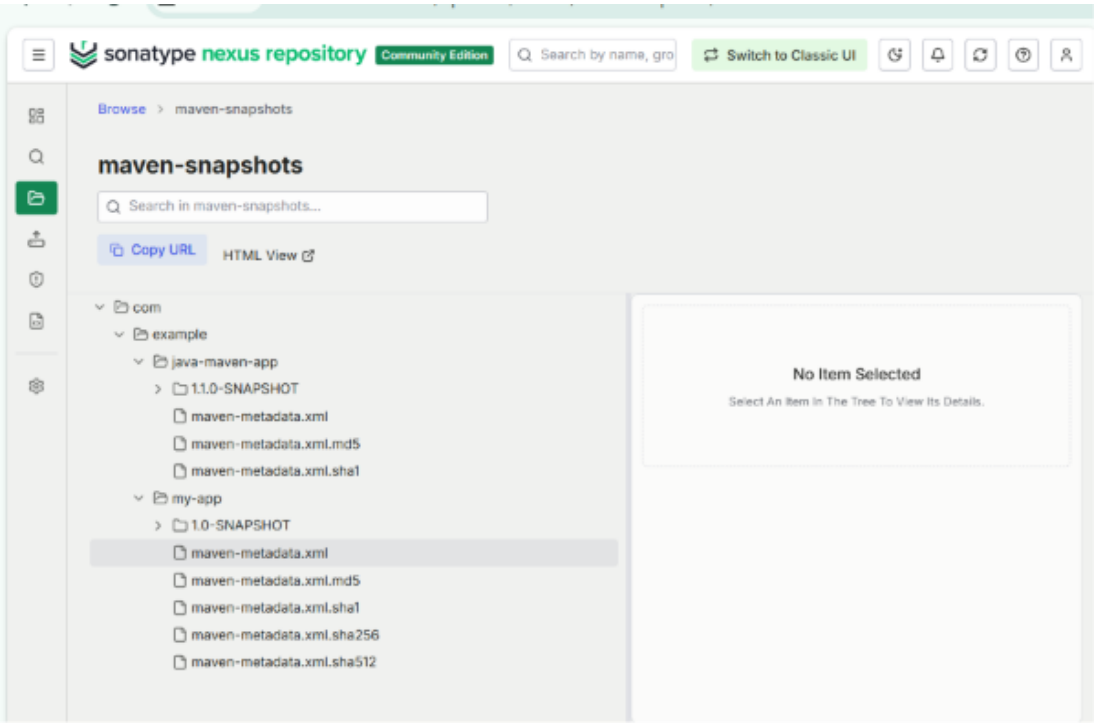


Figure 24. *nexus_maven_snapshots_empty.png* — maven-snapshots is empty after cleanup.

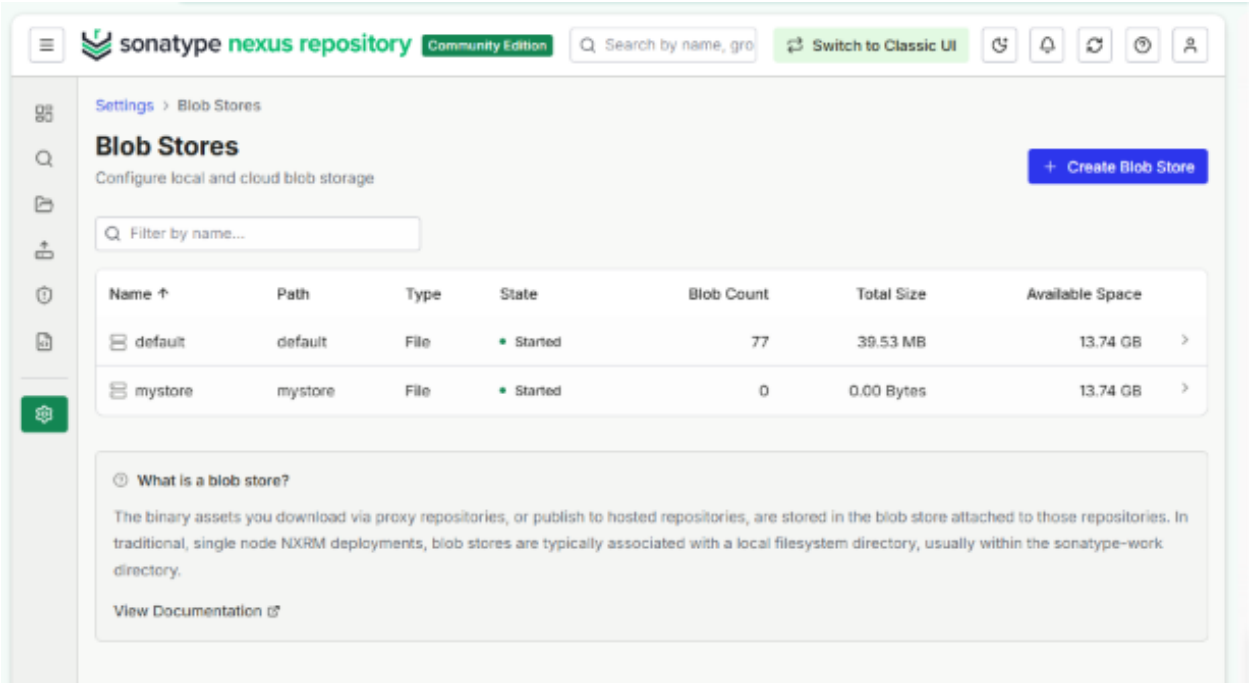


Figure 25. *nexus_blob_store_before_compaction.png* — Default blob store: blob count, used size and available space before compaction.

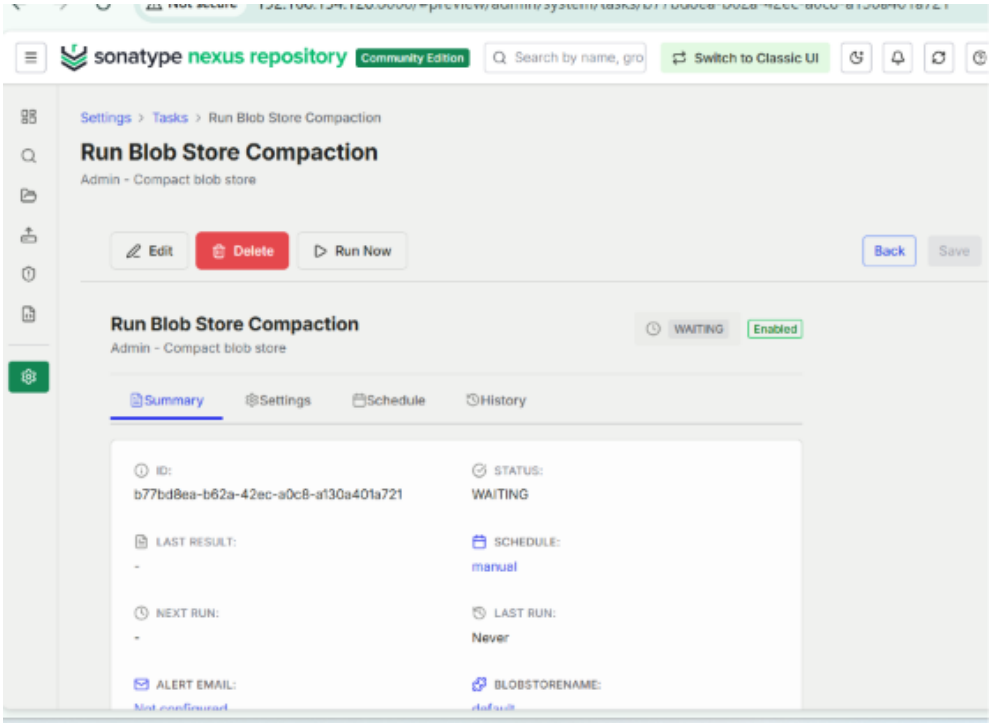


Figure 26. *nexus_compact_task_configured.png* — compact-store task: default blob store, 0 days, daily at 06:30.

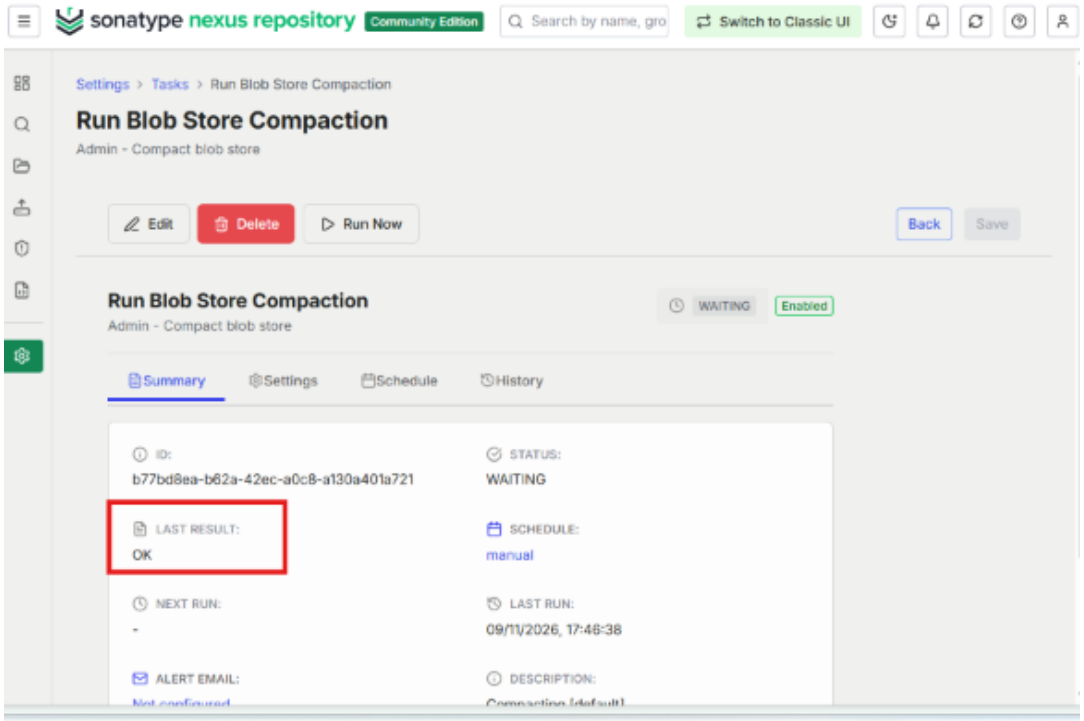


Figure 27. *nexus_compact_task_completed.png* — compact-store after the manual run: Waiting, Last result: Ok.

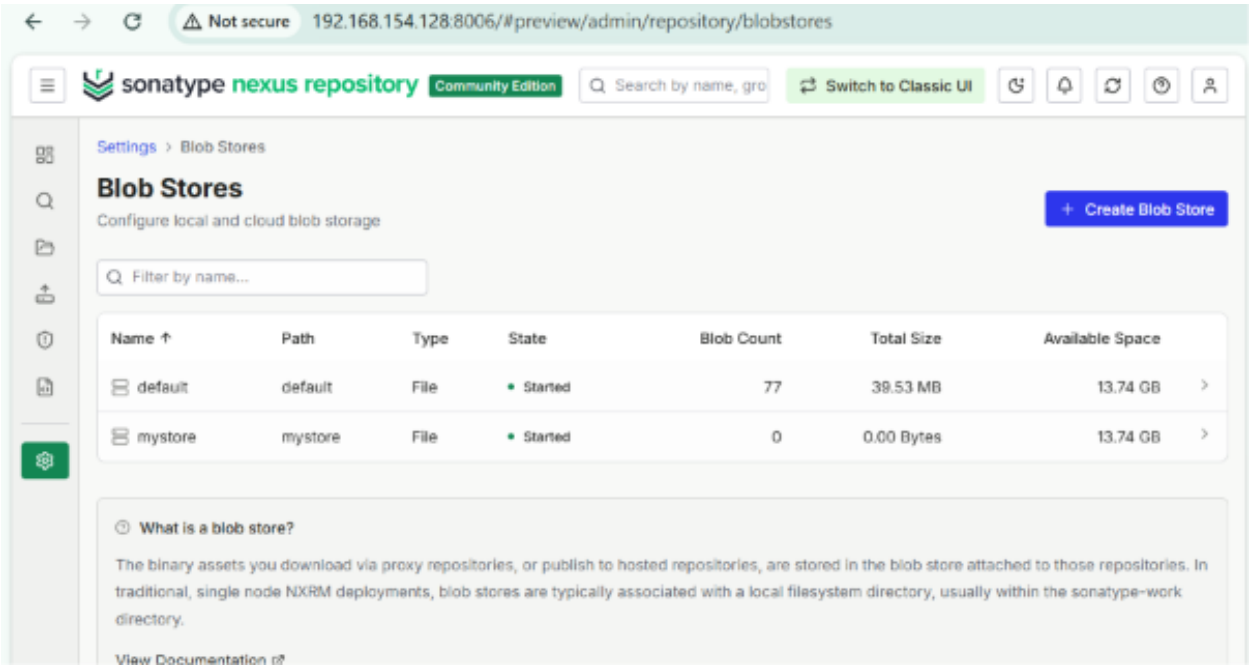


Figure 28. *nexus_blob_store_after_compaction.png* — Default blob store after compaction with reduced blob count/used size and more available space.